TRANSFORMER CALCULATION

Token	Pos	TokenEmb	Sinusoidal PE	Learned PE	RoPE(token)	Combined
the	0	[0.100,0.000,0.200]	[0.000,1.000,0.000]	[0.050,-0.020,0.010]	[0.100,0.000,0.200]	[0.100,1.000,0.200]
cat	1	[1.000,0.500,-0.200]	[0.841,0.540,0.002]	[0.020,0.030,-0.010]	[0.638,0.918,-0.200]	[1.841,1.040,-0.198]
sat	2	[0.200,0.900,0.100]	[0.909,-0.416,0.004]	[-0.030,0.010,0.020]	[-0.649,0.655,0.100]	[1.109,0.484,0.104]

Q=Combined	K=Combined	V=Combined	Q·K(the)	Q·K(cat)	Q·K(sat)	Scaled logits (the, cat, sat)
[0.100,1.000,0.200]	[0.100,1.000,0.200]	[0.100,1.000,0.200]	1.185			0.684
[1.841,1.040,-0.198]	[1.841,1.040,-0.198]	[1.841,1.040,-0.198]		4.512		2.605
[1.109,0.484,0.104]	[1.109,0.484,0.104]	[1.109,0.484,0.104]			2.525	1.458

Softmax weights (the, cat, sat)	AttnOut (cat)	FFN out (cat)	LM_logits (the,cat,sat)	LM_probs (the,cat,sat)	LM_pred
0.1					
0.683	[1.508,0.916,-0.093]	[0.408,0.260,0.031]	[0.393200,0.266700,0.189900]	[0.370831,0.326672,0.302498]	the
0.217					

Token Embeddings:

Raw Data (corpus): One dimensional sequence of text

|Viewing Single Post From: Spoilers for the Week of February 11th|\n|Lil||Feb 1 2013, 09:58 AM|\nDon't care about Chloe/Taniel/Jen-Jen. Don't care about Sami, really, but hoping that we get some good "SAMANTHA GENE!!" Marlena Death-Stares out of it. And "newfound" feelings. Please. If only.\nSTEFANO!! STEFANO, STEFANO, STEFANO!!!! :cheer:\n|Spoilers for the Week of February 11th · DAYS: News, Spoilers & Discussion| *sigh* Fundamentalist community, let me pass on some advice to you I learned from the atheistic community:\nIf you have set yourself on fire, do not run.\nOkay? Okay?? Please?\nLook, D, you had two months to say to Harvard in private emails, "Im sorry, I shouldnt have been using that animation in my paid presentations. I wont use it again. I really do like 'Inner Life', though, and would love to use it in classroom presentations, from the BioVisions site, if that is acceptable."\nI sat here, for two months, waiting for that to happen, anything to happen, and it didnt. Two months, on your own terms, you could have written a similar post to yesterdays. I would have given you the benefit of the doubt-- maybe you didnt know the credits werent visible to the audience, and I wouldnt have said a word beyond this, as its Harvards problem, not mine. This would have been a funny joke to those of us involved in dealing with you people, but it would have been a PR non-issue for you.\nBut after you set yourself on fire, you didnt douse it out with a bucket of ice cold reality and accountability. You ran. And youre still running.\nWhy not just state "I screwed up. Sorry everyone." and move on? Why the excuses? Why the denial? Why the passive language? Why the vague words and cryptic capitalizations? Why the writes and rewrites of your 'press release'? We know it wasnt written of your own volition, or it would have been done *before* Harvard had to take action. And, your behavior before this, regarding this issue, is not indicative of someone who made an innocent mistake. Its weird.\nSo what with this frantic running? Is the inability to say "I was wrong" a pathological feature of Creationists? Or are you hiding something? Or is it both? Or is it more?\nAnd now we get Carey weighing in on the

- Bits ⇒ If we apply encoding on this Text (like using UTF-8), it turned into as follows

```
0111110001010110011010010110010101110111011010010110111001100111001000000101001101101001011011100110011101101100011001010010000000101000001101111011100110111010000100000010
0110011100100110111101101101001110100010000000101001101110000011011110110100101101100011001010111001001110011001000000110011001101111011100100010000001110100011010000110010
0010000001010111011001010110010101101011001000000110111101100110001000000100011001100101011000100111001001110101011000010111001001111001001000000011000100110001011101000110
1000011111100010111000110111001111100010011000110100101101100011111000111110001000110011001010110001000100000001100010010000000110010001100000011000100110011001011000100000
0011000000111001001110100011010100111000001000000100000101001101011111000101110001101110010001000110111101101110001001110111010000100000011000110110000101110010011001010010
0000011000010110001001101111011101010111010000100000010000110110100001101100011011110110010100101111010101000110000101101110011010010110010101101100001011110100101001100101
0110111000101101010010100110010101101110001011100010000001000100011011110110111000100111011101000010000001100011011000010111001001100101001000000110000101100010011011110111
0101011101000010000001010011011000010110110101101001001011000010000001110010011001010110000101101100011011000111100100101100001000000110001001110101011101000010000001101000
0110111101110000011010010110111001100111001000000111010001101000011000010111010000100000011101110110010100100000011001110110010101110100001000000111001101101111011011010110
0101001000000110011101101111011011110110010000100000010001001010011010000010100110101000001010011100101010001000010000010010000010001110100010101001110010001010010000001
0010000100100010001000000100110101100001011100100110110001100101011011100110000100100000010001000110010101100001011101000110100001011010100110111010001100001011100100110
0101011100110010000001101111011101010111010000100000011011110110011000100000011010010111010000101110001000000100000101101110011001000010000001000100110111001100101011110111
0110011001101111011101010110111001100100001000100010000001100110011001010110010101101100011010010110111001100111011100110010111000100000010100000110110001100101011000010111
0011011001010010111000100000010010010110011000010000001101111011011100110110000111100100101110010111000110111001010011010101000100010101000110010000010100111001001111001000001
```

Byte Pair Algorithm

124 86 105 101 119 105 110 103 32 83 105 110 103 108 101 32 80 111 115 116 32 70 114 111 109 58 32 83 112 111 116 32 45/94 91 32 87 101
101 107 32 111 102 32 70 101 98 114 117 97 114 121 32 49 49 116 104 124 92 110 124 76 105 108 124 124 70 101 9 55 77 124 92
110 68 111 110 39 116 32 99 97 114 101 32 97 98 111 117 116 32 67 104 108 111 101 47 84 97 110 105 101 108 47 74 101 110 45 74 101 110 46 32 68 111 110 39 116 32 99 97 114
101 32 97 98 111 117 116 32 83 97 109 105 44 32 114 101 97 108 108 121 44 32 98 117 116 32 104 111 112 105 110 103 32 116 104 97 116 32 119 101 32 103 101 116 32 115 111
109 101 32 103 111 111 100 32 34 83 65 77 65 78 84 72 65 32 71 69 78 69 33 33 34 32 77 97 114 108 101 110 97 32 68 101 97 116 104 45 83 116 97 114 101 115 32 111 117 116 32
111 102 32 105 116 46 32 65 110 100 32 34 110 101 119 102 111 117 110 100 34 32 102 101 101 108 105 110 103 115 46 32 80 108 101 97 115 101 46 32 73 102 32 111 110 108 121
46 92 110 83 84 69 70 65 78 79 33 33 32 83 84 69 70 65 78 79 44 32 83 84 69 70 65 78 79 44 32 83 84 69 70 65 78 79 33 33 33 33 32 58 99 104 101 101 114 58 92 110 124 83 112
111 105 108 101 114 115 32 102 111 114 32 116 104 101 32 87 101 101 107 32 111 102 32 70 101 98 114 117 97 114 121 32 49 49 116 104 32 194 183 32 68 65 89 83 58 32 78 101
119 115 44 32 83 112 111 105 108 101 114 115 32 38 32 68 105 115 99 117 115 115 105 111 110 124 32 42 115 105 103 104 42 32 70 117 110 100 97 109 101 110 116 97 108 105 115
116 32 99 111 109 109 117 110 105 116 121 44 32 108 101 116 32 109 101 32 112 97 115 115 32 111 110 32 115 111 109 101 32 97 100 118 105 99 101 32 116 111 32 121 111 117 32
73 32 108 101 97 114 110 101 100 32 102 114 111 109 32 116 104 101 32 97 116 104 101 105 115 116 105 99 32 99 111 109 109 117 110 105 116 121 58 92 110 73 102 32 121 111
117 32 104 97 118 101 32 115 101 116 32 121 111 117 114 115 101 108 102 32 111 110 32 102 105 114 101 44 32 100 111 32 110 111 116 32 114 117 110 46 92 110 79 107 97 121 63
32 79 107 97 121 63 63 32 80 108 101 97 115 101 63 92 110 76 111 111 107 44 32 68 44 32 121 111 117 32 104 97 100 32 116 119 111 32 109 111 110 116 104 115 32 116 111 32
115 97 121 32 116 111 32 72 97 114 118 97 114 100 32 105 110 32 112 114 105 118 97 116 101 32 101 109 97 105 108 115 44 32 34 73 109 32 115 111 114 114 121 44 32 73 32 115
104 111 117 108 100 110 116 32 104 97 118 101 32 98 101 101 110 32 117 115 105 110 103 32 116 104 97 116 32 97 110 105 109 97 116 105 111 110 32 105 110 32 109 121 32 112
97 105 100 32 112 114 101 115 101 110 116 97 116 105 111 110 115 46 32 73 32 119 111 110 116 32 117 115 101 32 105 116 32 97 103 97 105 110 46 32 73 32 114 101 97 108 108

	battle	horse	king	man	queen	..	woman
authority	0	0.01	1	0.2	1		0.2
event	1	0	0	0	0		0
has tail?	0	1	0	0	0		0
rich	0	0.1	1	0.3	1		0.2
gender	0	1	-1	-1	1		1

Positional Embedding

Sinusoidal PE

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right), \quad PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

Method	Static/Learned	Example Models
Sinusoidal (absolute)	Static, formula-based	Transformer (2017), T5 (relative bias variant)
Learned absolute	Neural network learns embeddings	BERT, GPT-2, GPT-3, RoBERTa
Relative (bias/encoding)	Learned, distance-aware	Transformer-XL, T5, DeBERTa
RoPE (rotary)	Deterministic rotations, trainable with tokens	LLaMA, GPT-NeoX, Falcon, Qwen, MPT, Phi-2/3

3. Simple calculation example (sinusoidal PE)

Let's compute sinusoidal embeddings with:

- Embedding dimension `d_model = 4`
- Positions `pos = 0, 1, 2`

Step 1. Compute denominator for each dimension

- For `i=0` → denominator = (10000^{0/4} = 1)
- For `i=1` → denominator = (10000^{2/4} = 100)

Step 2. Compute PE values

pos	PE(pos,0) = sin(pos/1)	PE(pos,1) = cos(pos/1)	PE(pos,2) = sin(pos/100)	PE(pos,3) = cos(pos/100)
0	sin(0) = 0.0000	cos(0) = 1.0000	sin(0/100) = 0.0000	cos(0/100) = 1.0000
1	sin(1) = 0.8415	cos(1) = 0.5403	sin(0.01) = 0.0100	cos(0.01) = 0.9999
2	sin(2) = 0.9093	cos(2) = -0.4161	sin(0.02) = 0.0200	cos(0.02) = 0.9998

So:

- `PE(0) = [0.0000, 1.0000, 0.0000, 1.0000]`
- `PE(1) = [0.8415, 0.5403, 0.0100, 0.9999]`
- `PE(2) = [0.9093, -0.4161, 0.0200, 0.9998]`

Why "i" is half of d_model size? For d_model=4 --> i = 2 For d_model=8 --> i = 4

Example: `d_model = 4`

- For `i = 0`: fill dims 0 and 1 → denominators = $10000^{0/2} = 1$
- For `i = 1`: fill dims 2 and 3 → denominators = $10000^{2/4} = 100$

So for each position, we calculate 4 values:

$$[\sin(pos/1), \cos(pos/1), \sin(pos/100), \cos(pos/100)]$$

Correction: 10000^(0/4) not 10000^(0/2)

Example: `d_model = 8`

- `i = 0` → dims 0,1 → denominator = $10000^{0/8} = 1$
- `i = 1` → dims 2,3 → denominator = $10000^{2/8} = 10$
- `i = 2` → dims 4,5 → denominator = $10000^{4/8} = 100$
- `i = 3` → dims 6,7 → denominator = $10000^{6/8} = 1000$

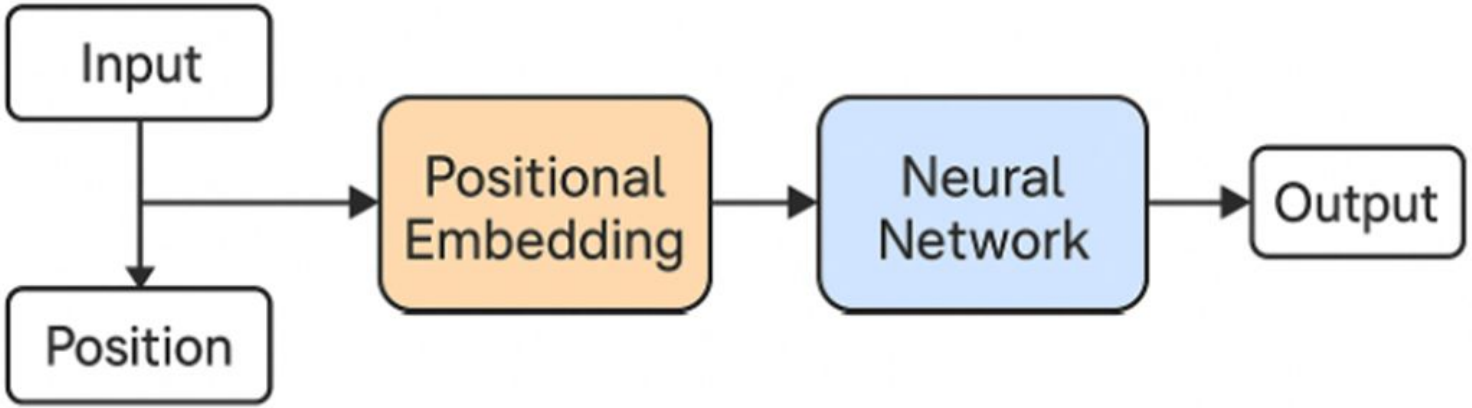
So each position gets:

$$[\sin(pos/1), \cos(pos/1), \sin(pos/10), \cos(pos/10), \sin(pos/100), \cos(pos/100), \sin(pos/1000), \cos(pos/1000)]$$

- ✅ So **pos** walks over tokens in the input sequence.
- ✅ **i** walks over embedding dimensions (half as many as `d_model`, because sin/cos come in pairs).

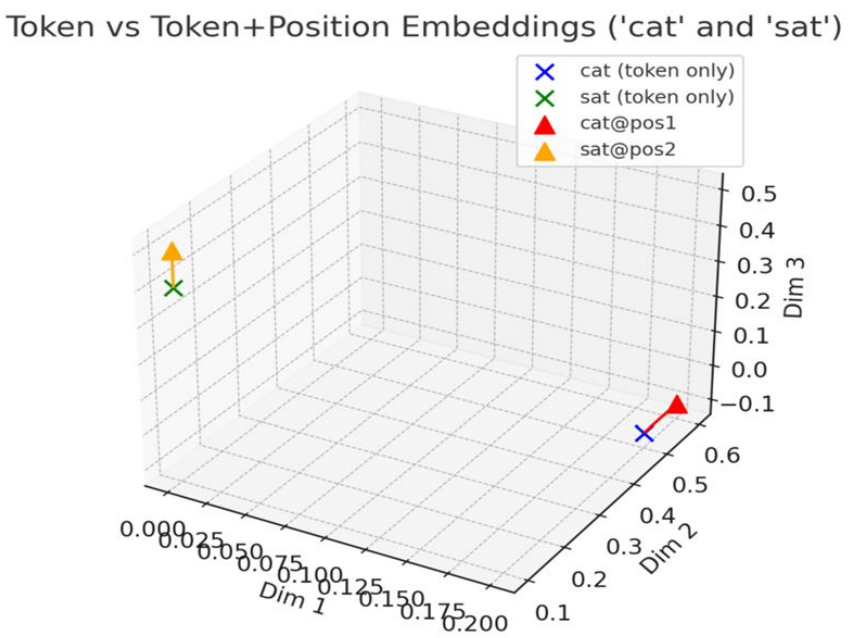
Learned PE

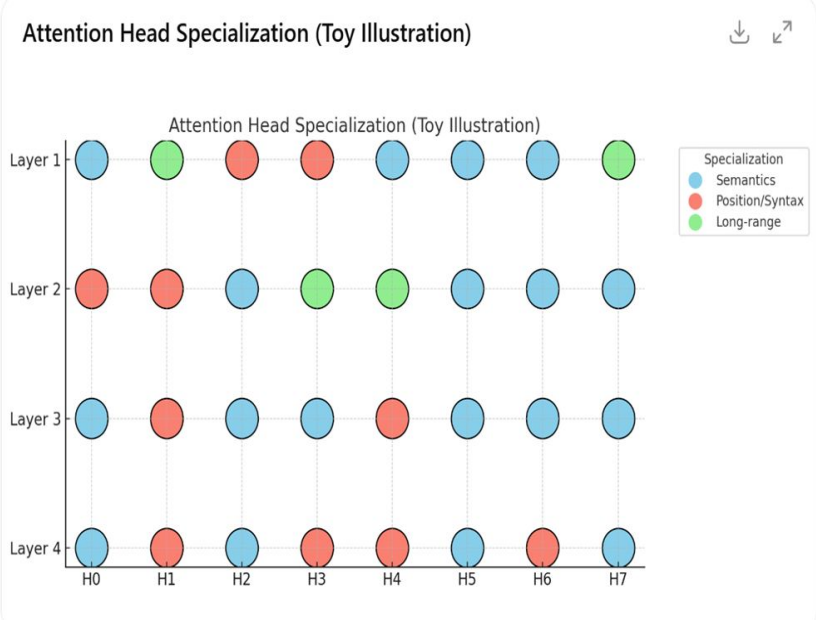
Just like the way, B0 and B1 calculated in Gradient Descent, we learned, Each value in Positional Embedding array going to be considered as Parameter and will be calculated based on Error and recaliberation based on Error



RoPE (Rotatory Positional Embedding)

Transformer Architecture learns the position of Embeddings by adding ROTATION by Trail and Error and thereafter Recaliberation rotation as a parameter. Some of the Attention Heads in certain Transformer based Architectures, focusses solely on Rotation





Rotary Position Embeddings (RoPE, used in GPTNeoX, LLaMA, Qwen2.5)

Don't store a separate table.

Instead, apply a **rotation in the complex plane** to queries and keys:

$$Q'_i = Q_i \cdot R(i), \quad K'_j = K_j \cdot R(j)$$

where $R(\cdot)$ is a rotation matrix.

Then the dot product $Q'_i \cdot K'_j$ inherently encodes relative distance $(i - j)$.

Relative Positional Embedding

(a) Learned Relative Position Embeddings (Transformer-XL, T5, DeBERTa)

- Define a **table of embeddings** R indexed by relative positions.

  Example: if max relative distance = 4, then $R = \{-4, -3, -2, -1, 0, 1, 2, 3, 4\}$.
- For attention between token i and token j:
  - Compute $Q_i \cdot K_j + Q_i \cdot R_{i-j}$.
- R_{i-j} is trainable.

(b) Relative Bias Matrices (T5-style)

- Instead of full vectors, use a **bias scalar** per relative distance:

$$\text{Score}(i, j) = Q_i \cdot K_j + b_{i-j}$$

- This is simpler and cheaper (used in T5).

4. Example (T5-style relative bias, max distance = 2)

Suppose we attend over a sequence of length 4: positions = [0,1,2,3].

Relative distances between tokens:

i (query)	j (key)	i-j	bias = b[i-j]
0	0	0	b[0]
0	1	-1	b[-1]
0	2	-2	b[-2]
1	0	1	b[+1]
2	0	2	b[+2]

Each attention score is:

$$\text{Score}(i, j) = Q_i \cdot K_j + b[i - j]$$

✅ **Summary:**

- **Absolute PE** = add sin/cos(position) or learned vector to input.
- **Relative PE** = adjust *attention score* with an embedding or bias depending on **distance (i-j)**.
- Implementations differ:
  - Transformer-XL/DeBERTa: learned vectors R_{i-j}
  - T5: learned scalar biases $b[i - j]$
  - RoPE: implicit encoding via rotations.

1. What happens with absolute (sinusoidal or learned) positional embeddings?

- Each token is tagged with an **absolute coordinate** (0,1,2,...).
- If you **pad at the start** (or otherwise shift your sequence), then:
  - The token that *used to be at position 0* is now at, say, position 5.
  - Its positional encoding changes drastically (different sin/cos pattern).
- But its **semantic relation** to the rest of the sentence hasn't changed — only the absolute index has.
- The model has to *re-learn* or adapt to every possible shift.

---

2. Why does this make sinusoidal PEs "stale" with padding?

Padding = inserting meaningless tokens at the front.

- In practice, NLP pipelines often pad sequences to the same length for batching.
- Example:
  - Original: `[The, cat, sat]` → positions `[0,1,2]`
  - Padded: `[PAD, PAD, The, cat, sat]` → positions `[0,1,2,3,4]`

Now:

- "The" was position 0 → becomes position 2.
- Its PE vector changes.
- Absolute PE **cannot tell** that the true relation between "The" and "cat" (distance = 1) is unchanged.

Method	Static/Learned	Example Models
Sinusoidal (absolute)	Static, formula-based	Transformer (2017), T5 (relative bias variant)
Learned absolute	Neural network learns embeddings	BERT, GPT-2, GPT-3, RoBERTa
Relative (bias/encoding)	Learned, distance-aware	Transformer-XL, T5, DeBERTa
RoPE (rotary)	Deterministic rotations, trainable with tokens	LLaMA, GPT-NeoX, Falcon, Qwen, MPT, Phi-2/3

Combined Embedding

We can choose any embedding depends upon the type of the Model

2) Combined embeddings (Token + Sinusoidal PE)

Token	Pos	Token Emb	Sinusoidal PE	Combined
the	0	[0.100000, 0.000000, 0.200000]	[0.000000, 1.000000, 0.000000]	[0.100000, 1.000000, 0.200000]
cat	1	[1.000000, 0.500000, -0.200000]	[0.841471, 0.540302, 0.002154]	[1.841471, 1.040302, -0.197846]
sat	2	[0.200000, 0.900000, 0.100000]	[0.909297, -0.416147, 0.004309]	[1.109297, 0.483853, 0.104309]

+

TokenEmb	Sinusoidal PE	Learned PE	RoPE(token)	Combined
[0.100,0.000,0.200]	[0.000,1.000,0.000]	[0.050,-0.020,0.010]	[0.100,0.000,0.200]	[0.100,1.000,0.200]
[1.000,0.500,-0.200]	[0.841,0.540,0.002]	[0.020,0.030,-0.010]	[0.638,0.918,-0.200]	[1.841,1.040,-0.198]
[0.200,0.900,0.100]	[0.909,-0.416,0.004]	[-0.030,0.010,0.020]	[-0.649,0.655,0.100]	[1.109,0.484,0.104]

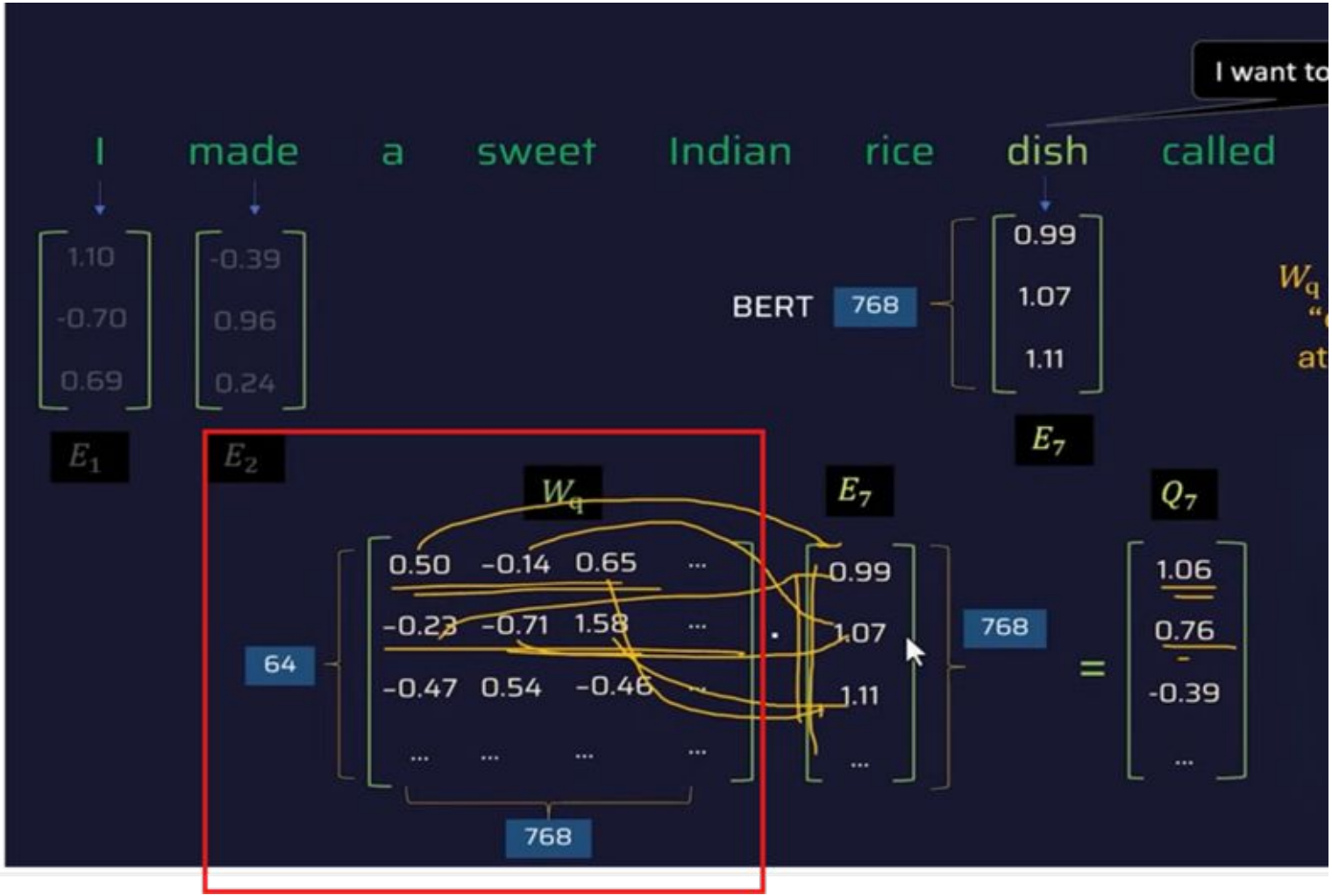
Q, K, V

Here we have Taken Q, K and V as Identity Matrix. So, Q . Coombined_Matrix (dot product of Q and Combined Matrix) is giving same as Combined Matrix.

But, in Real Word, a Pretrained Model comes with its own Q, K and V weights

Here we have taken Identity Matrix

G	H	I	J
Combined	Q=Combined	K=Combined	V=Combined
[0.100,1.000,0.200]	[0.100,1.000,0.200]	[0.100,1.000,0.200]	[0.100,1.000,0.200]
[1.841,1.040,-0.198]	[1.841,1.040,-0.198]	[1.841,1.040,-0.198]	[1.841,1.040,-0.198]
[1.109,0.484,0.104]	[1.109,0.484,0.104]	[1.109,0.484,0.104]	[1.109,0.484,0.104]


I want to
I made a sweet Indian rice dish called
1.10 -0.70 0.69
-0.39 0.96 0.24
BERT 768
0.99 1.07 1.11
E_1
E_2
E_7
W_q
E_7
Q_7
0.50 -0.14 0.65 ...
-0.23 -0.71 1.58 ...
-0.47 0.54 -0.46 ...
... ...
64
768
0.99 1.07 1.11 ...
768
1.06 0.76 -0.39 ...

key

I am the subject of the sentence

I made a sweet Indian rice dish called

$$E_1 = \begin{bmatrix} 1.10 \\ -0.70 \\ 0.69 \end{bmatrix}$$

W_k knows how to encode "key" of a token for attention computation

$$\underbrace{W_k}_{64 \times 768} \cdot \underbrace{E_1}_{768 \times 1} = K_1$$

$$W_k = \begin{bmatrix} 0.28 & 1.61 & 0.23 & \cdots \\ 0.19 & 0.55 & 0.37 & \cdots \\ 0.97 & 1.01 & -0.19 & \cdots \\ \cdots & \cdots & \cdots & \cdots \end{bmatrix}, \quad E_1 = \begin{bmatrix} 1.10 \\ -0.70 \\ 0.69 \\ \cdots \end{bmatrix}, \quad K_1 = \begin{bmatrix} 0.31 \\ -1.12 \\ 0.16 \\ \cdots \end{bmatrix}$$

key

I am the subject of the sentence

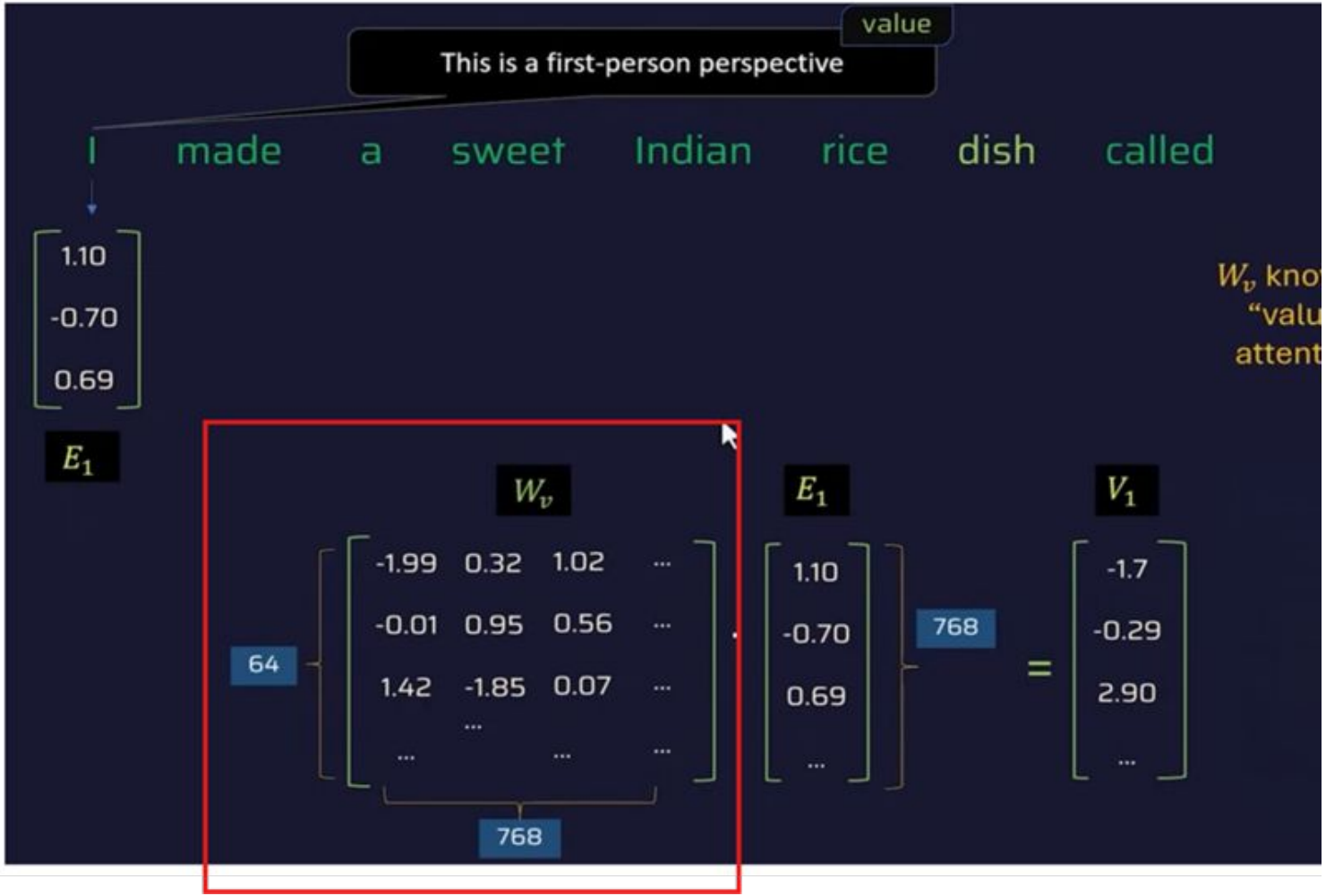
I made a sweet Indian rice dish called

$$E_1 = \begin{bmatrix} 1.10 \\ -0.70 \\ 0.69 \end{bmatrix}$$

W_k knows how to encode "key" of a token for attention computation

$$\underbrace{W_k}_{64 \times 768} \cdot \underbrace{E_1}_{768 \times 1} = K_1$$

$$W_k = \begin{bmatrix} 0.28 & 1.61 & 0.23 & \cdots \\ 0.19 & 0.55 & 0.37 & \cdots \\ 0.97 & 1.01 & -0.19 & \cdots \\ \cdots & \cdots & \cdots & \cdots \end{bmatrix}, \quad E_1 = \begin{bmatrix} 1.10 \\ -0.70 \\ 0.69 \\ \cdots \end{bmatrix}, \quad K_1 = \begin{bmatrix} 0.31 \\ -1.12 \\ 0.16 \\ \cdots \end{bmatrix}$$


value
This is a first-person perspective
I made a sweet Indian rice dish called
1.10
-0.70
0.69
E_1
W_v kno
"valu
attent
W_v
-1.99 0.32 1.02 ...
-0.01 0.95 0.56 ...
1.42 -1.85 0.07 ...
...
64
768
E_1
1.10
-0.70
0.69
...
768
=
V_1
-1.7
-0.29
2.90
...

Q.K (Relative to "cat")

For Simplicity, we are calculating only Q_cat * other Ks i.e., K_cat, K_the, K_sat. Not Others

$$Attention(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

H	I	J	K	L	M	N
Q=Combined	K=Combined	V=Combined	Q·K(the)	Q·K(cat)	Q·K(sat)	Scaled logits (the, cat, sat)
[0.100,1.000,0.200]	[0.100,1.000,0.200]	[0.100,1.000,0.200]	1.185			0.684
[1.841,1.040,-0.198]	[1.841,1.040,-0.198]	[1.841,1.040,-0.198]		4.512		2.605
[1.109,0.484,0.104]	[1.109,0.484,0.104]	[1.109,0.484,0.104]			2.525	1.458

3) Q, K, V projections (identity used)

Token	Q	K	V
the	[0.100000, 1.000000, 0.200000]	[0.100000, 1.000000, 0.200000]	[0.100000, 1.000000, 0.200000]
cat	[1.841471, 1.040302, -0.197846]	[1.841471, 1.040302, -0.197846]	[1.841471, 1.040302, -0.197846]
sat	[1.109297, 0.483853, 0.104309]	[1.109297, 0.483853, 0.104309]	[1.109297, 0.483853, 0.104309]

---

4) Attention calculation (query = cat @ pos1)

4.1 Dot products (Q_cat · K_j)

- Q_cat · K_the
  `(1.841471*0.100000) + (1.040302*1.000000) + (-0.197846*0.200000)`
  `= 0.184147 + 1.040302 - 0.039569 = 1.184880`
- Q_cat · K_cat
  `(1.841471^2) + (1.040302^2) + (-0.197846^2)`
  `= 3.391012 + 1.081028 + 0.040347 = 4.512387`
- Q_cat · K_sat
  `(1.841471*1.109297) + (1.040302*0.483853) + (-0.197846*0.104309)`
  `= 2.043600 + 0.503395 - 0.021541 = 2.525456`

Key	Q·K
the	1.184880
cat	4.512387
sat	2.525456

4.2 Scale by √d_k (√3 = 1.732051)

Key	Scaled logit
the	0.684091
cat	2.605228
sat	1.458072

Softmax

$$Attention(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Softmax Calculation (example for logit = 0.684)

We compute softmax over the three **scaled logits**:

`[0.684, 2.605, 1.458]`

Step 1: Exponentiate each value

Formula: `e^x = exp(x)`

- `exp(0.684) ≈ 1.9819`
- `exp(2.605) ≈ 13.5309`
- `exp(1.458) ≈ 4.2982`

Step 2: Sum of exponentials

`Sum = 1.9819 + 13.5309 + 4.2982 = 19.8110`

Step 3: Compute softmax weights

Formula: `Softmax(x_i) = exp(x_i) / Sum`

- For **0.684**: `1.9819 / 19.8110 = 0.1000`
- For **2.605**: `13.5309 / 19.8110 = 0.6831`
- For **1.458**: `4.2982 / 19.8110 = 0.2169`

	O
	Softmax weights (the, cat, sat)
4	0.1
5	0.683
3	0.217

Attention ("cat")

For simplicity, we are calculating Attention only for "cat"

	P
	AttnOut (cat)
1	
3	[1.508,0.916,-0.093]
7	

J
V=Combined
[0.100,1.000,0.200]
[1.841,1.040,-0.198]
[1.109,0.484,0.104]

softmax * V

Step 1: Recall the softmax weights (from 4.3)

- the → 0.100029
- cat → 0.683070
- sat → 0.216901

So:

$$\alpha_{\text{the}} = 0.100029, \quad \alpha_{\text{cat}} = 0.683070, \quad \alpha_{\text{sat}} = 0.216901$$

Step 2: Recall the V vectors (from section 3)

- V_the = `[0.100000, 1.000000, 0.200000]`
- V_cat = `[1.841471, 1.040302, -0.197846]`
- V_sat = `[1.109297, 0.483853, 0.104309]`

Step 3: Compute weighted sum

$$\text{AttnOut}_{cat} = \alpha_{\text{the}} \cdot V_{\text{the}} + \alpha_{\text{cat}} \cdot V_{\text{cat}} + \alpha_{\text{sat}} \cdot V_{\text{sat}}$$

Component 1 (first dimension)

$$= (0.100029 \times 0.100000) + (0.683070 \times 1.841471) + (0.216901 \times 1.109297)$$
$$= 0.010003 + 1.257889 + 0.240572 = 1.508464$$

Component 2 (second dimension)

$$= (0.100029 \times 1.000000) + (0.683070 \times 1.040302) + (0.216901 \times 0.483853)$$
$$= 0.100029 + 0.710777 + 0.104770 = 0.915576$$

Component 3 (third dimension)

$$= (0.100029 \times 0.200000) + (0.683070 \times -0.197846) + (0.216901 \times 0.104309)$$
$$= 0.020006 - 0.135131 + 0.022613 = -0.092512$$

✅ **Final result**

$$\text{AttnOut}_{cat} = [1.508464,\ 0.915576,\ -0.092512]$$

$$\text{AttnOut}(\text{cat}) = (\alpha_{\text{cat}\to\text{the}} \cdot V_{\text{the}}) + (\alpha_{\text{cat}\to\text{cat}} \cdot V_{\text{cat}}) + (\alpha_{\text{cat}\to\text{sat}} \cdot V_{\text{sat}})$$

Step 1: Arrange the value vectors

Stack the values (V_{the}, V_{cat}, V_{sat}) as rows in a matrix:

$$V = \begin{bmatrix} 0.100000 & 1.000000 & 0.200000 \\ 1.841471 & 1.040302 & -0.197846 \\ 1.109297 & 0.483853 & 0.104309 \end{bmatrix}$$

This is a (3 \times 3) matrix (3 tokens × 3 dimensions).

Step 2: Write the attention weights

The attention distribution for query = *"cat"* is a row vector:

$$\alpha_{cat} = \begin{bmatrix} 0.100029 & 0.683070 & 0.216901 \end{bmatrix}$$

Step 3: Multiply

The attention output is:

$$\text{AttnOut}_{cat} = \alpha_{cat} \times V$$

That is, a (1 \times 3) vector:

$$\begin{bmatrix} 0.100029 & 0.683070 & 0.216901 \end{bmatrix} \begin{bmatrix} 0.100000 & 1.000000 & 0.200000 \\ 1.841471 & 1.040302 & -0.197846 \\ 1.109297 & 0.483853 & 0.104309 \end{bmatrix} = \begin{bmatrix} 1.508464 & 0.915576 & -0.092512 \end{bmatrix}$$

Key takeaway

So yes — the attention output is exactly a **weighted average of the value vectors**, implemented as:

$$\text{AttnOut}_q = \alpha_q V$$

- (\alpha_q) = attention weights for query (q) (row vector, length = number of tokens)
- (V) = matrix of value vectors (tokens × hidden_dim)
- Result = single vector of dimension = hidden_dim

Final Part

P	Q	R	S	T
AttnOut (cat)	FFN out (cat)	LM_logits (the,cat,sat)	LM_probs (the,cat,sat)	LM_pred
[1.508,0.916,-0.093]	[0.408,0.260,0.031]	[0.393200,0.266700,0.189900]	[0.370831,0.326672,0.302498]	the

Step 1: First linear layer (W1)

We want `W1 * a = h_pre`.

```
W1 ≈ [[ 0.3646,  0.2212, -0.0223],
      [ 0.2213,  0.1342, -0.0136],
      [-0.0224, -0.0136,  0.0014],
      [ 0.1479,  0.0898, -0.0091]]
b1 = [0, 0, 0, 0]
```

Row-by-row checks

- Row 1:
  `(0.3646*1.508464) + (0.2212*0.915576) + (-0.0223*-0.092512)`
  `= 0.5499 + 0.2021 + 0.0022 = 0.7542 ≈ h_pre[0]`
- Row 2:
  `(0.2213*1.508464) + (0.1342*0.915576) + (-0.0136*-0.092512)`
  `= 0.3337 + 0.1229 + 0.0013 = 0.4579 ≈ h_pre[1]`
- Row 3:
  `(-0.0224*1.508464) + (-0.0136*0.915576) + (0.0014*-0.092512)`
  `= -0.0338 + -0.0125 + -0.0001 = -0.0464 ≈ h_pre[2]`
- Row 4:
  `(0.1479*1.508464) + (0.0898*0.915576) + (-0.0091*-0.092512)`
  `= 0.2231 + 0.0822 + 0.0008 = 0.3061 ≈ h_pre[3]`

So indeed:

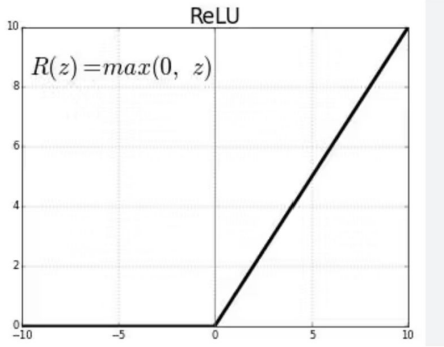
```
h_pre = [0.754232, 0.457788, -0.046256, 0.306208]
```

Step 2: Apply ReLU

```
h_relu = ReLU(h_pre)
       = [0.754232, 0.457788, 0.000000, 0.306208]
```

(negative third component becomes 0)

- Number of layers:
  - **2 linear (dense) layers** (which is exactly what a Transformer FFN block has).
    - Layer 1: `Linear(3 → 4)`
    - ReLU activation
    - Layer 2: `Linear(4 → 3)`
- Neurons (units):
  - **Layer 1 (W1):** 4 neurons (so output dimension is 4)
  - **Layer 2 (W2):** 3 neurons (so output dimension goes back to 3, matching the embedding size)



Step 3: Second linear layer (W2)

We want `W2 * h_relu = y`.

```
W2 ≈ [[0.3527, 0.2140, 0,      0.1432],
      [0.2242, 0.1362, 0,      0.0911],
      [0.0265, 0.0161, 0,      0.0108]]
b2 = [0, 0, 0]
```

Row-by-row checks

- Row 1:
  `(0.3527*0.754232) + (0.2140*0.457788) + (0*0.000000) + (0.1432*0.306208)`
  `= 0.2661 + 0.0980 + 0.0000 + 0.0436 = 0.4077 ≈ y[0]`
- Row 2:
  `(0.2242*0.754232) + (0.1362*0.457788) + (0*0.000000) + (0.0911*0.306208)`
  `= 0.1691 + 0.0624 + 0.0000 + 0.0279 = 0.2594 ≈ y[1]`
- Row 3:
  `(0.0265*0.754232) + (0.0161*0.457788) + (0*0.000000) + (0.0108*0.306208)`
  `= 0.0200 + 0.0074 + 0.0000 + 0.0033 = 0.0307 ≈ y[2]`

So:

```
y = [0.407737, 0.259515, 0.030621]
```

Summary (single-pass)

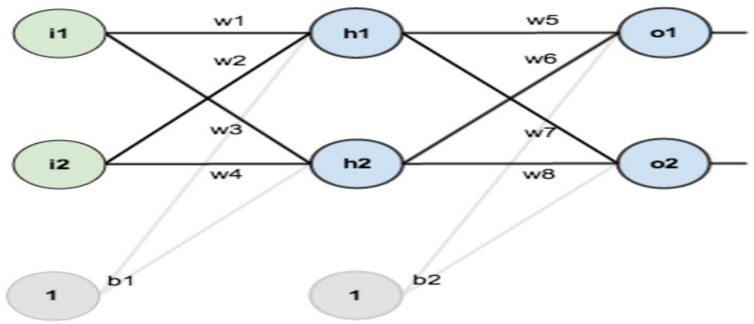
```
Input a = [1.508464, 0.915576, -0.092512]

Step 1: Linear    -> h_pre = W1 * a + b1 = [0.754232, 0.457788, -0.046256, 0.306208]
Step 2: ReLU      -> h_relu = [0.754232, 0.457788, 0.000000, 0.306208]
Step 3: Linear    -> y = W2 * h_relu + b2 = [0.407737, 0.259515, 0.030621]
```

$$W = W - \alpha \frac{\partial L}{\partial W}$$

$$b = b - \alpha \frac{\partial L}{\partial b} \qquad (\alpha - \textit{Learning Rate})$$

Chain rule: $F'(x) = \frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}$



Adjusted Weights to get the right Prediction

```
W_out ≈ [
  [ 0.00000000,  0.00000000,  0.00000000],   # for "the"
  [ 0.00000000,  0.00000000,  0.00000000],   # for "cat"
  [ 2.60397830,  1.65939794,  0.19785129]    # for "sat"
]
b_out = [0.0, 0.0, 0.0]
```

Compute logits = W_out @ h + b_out

Row-wise dot-products:

- logit_the = `[0,0,0] · h = 0.0`
- logit_cat = `[0,0,0] · h = 0.0`
- logit_sat = `[2.6039783, 1.65939794, 0.19785129] · [0.408,0.260,0.031] = 1.5`

So:

```
logits = [0.0, 0.0, 1.5]
```

Softmax over logits

Formula:

```
softmax(x)_i = exp(x_i) / sum_j exp(x_j)
```

Numeric:

- `exp(0.0) = 1.000000`
- `exp(0.0) = 1.000000`
- `exp(1.5) ≈ 4.481689`

Sum = `1 + 1 + 4.481689 = 6.481689`

Probs:

- P("the") = `1 / 6.481689 ≈ 0.154281`
- P("cat") = `1 / 6.481689 ≈ 0.154281`
- P("sat") = `4.481689 / 6.481689 ≈ 0.691438`

Final prediction

```
LM_pred = "sat"   (prob ≈ 0.6914)
```